

ИИ-ассистент для обработки клиентских заявок

Собираем рабочий MVP: от поступления заявки до сохранения результата и передачи человеку на подтверждение.

ЦЕЛЬ УРОКА

Научиться превращать набор знакомых нод в целостное решение под бизнес-задачу: принять заявку, проверить данные, получить структурированный результат от LLM, сохранить его и подготовить безопасное следующее действие.

Что вы соберёте

Рабочий MVP в n8n, который:

- 01 принимает тестовую заявку
- 02 проверяет обязательные поля
- 03 передаёт содержание заявки в LLM
- 04 получает ответ по заданной JSON-структуре
- 05 классифицирует по теме и приоритету
- 06 сохраняет данные и черновик ответа в NocoDB
- 07 уведомляет ответственного человека
- 08 останавливается на черновике — отправлять клиенту нечем

Маршрут решения

Шесть шагов на канвасе. Сборка промпта уехала внутрь ноды агента — отдельного шага для неё больше нет.

01

Приём заявки

webhook

02

Валидация

обязательные поля, ticket_id

03

ИИ-агент

YandexGPT + схема ответа

04

Бизнес-правила

нормализация, эскалация приоритета

05

Запись в NocoDB

данные и черновик

06

Уведомление в Telegram

ответственному

Постановка задачи и границы автоматизации

Что определяем на входе

Кто пользователь и какую проблему решает MVP

Вход, выход и критерий успешной обработки

Что делает LLM, а что остаётся обычным правилом

ПРИНЦИП

Модель предлагает решение, но не принимает критичное бизнес-решение самостоятельно.

Приоритет, названный моделью, правило может повысить, но не понизить.

Контракт входных данных

✓ **КОРРЕКТНАЯ** · поля `name` · `contact` · `message`

```
{  
  "name": "Иван Петров",  
  "contact": "ivan@mail.ru",  
  "message": "Второй месяц списывают 990 ₽,  
  хотя подписку я отменил."  
}
```

✗ **НЕКОРРЕКТНАЯ** — нет `name`; либо `contact` не похож ни на почту, ни на телефон; либо `message` короче 10 символов.

**ВАЛИДАЦИЯ ДЕЛАЕТ БОЛЬШЕ,
ЧЕМ ПРОВЕРКУ**

- нормализует контакт
- обрезает текст до 4000 символов
- генерирует `ticket_id`
- помечает попытки подмены инструкций

Схема ответа модели

LLM возвращает строго заданный JSON. Схема живёт в трёх местах: Output Parser, системное сообщение агента и Code-нода.

```
{
  "category": "billing",
  "priority": "high",
  "summary": "...",
  "missing_data": [],
  "draft_reply": "..."
}
```

category	тема заявки
priority	приоритет обработки
summary	краткая суть
missing_data	чего не хватает в заявке
draft_reply	черновик ответа клиенту

Почему модель — это нода OpenAI

Нативной ноды YandexGPT в n8n нет. Берём **OpenAI Chat Model** и перенастраиваем её на Яндекс.

01

OpenAI Chat Model

та самая нода с чужим логотипом
— это норма

02

Подмена Base URL

эндпоинт указывает на
API YandexGPT

03

Folder в имени модели

идентификатор каталога
зашивается в название модели

Проектирование workflow · фактические ноды

01 **Webhook**

02 **Code:** валидация и сборка входа для агента

03 **IF:** заявка валидна — две ветки, у каждой свой Respond

04 **ИИ-агент** + YandexGPT + Схема ответа

05 **Code:** бизнес-правила и текст уведомления

06 **NocoDB:** сохранить

07 **Telegram:** уведомить

08 **Respond to Webhook**

Сборка вместе с экспертом

ДВА УЗЛА ВЕТВЛЕНИЯ

IF: заявка валидна

невалидные заявки не доходят до агента

Подузлы агента

подключение Model и Output Parser к ИИ-агенту

МАППИНГ ДАННЫХ

```
$( 'Валидация заявки')  
  .item.json
```

Обращаемся к ноде по имени, а не к `$json`: так берём выход конкретного этапа независимо от того, что стоит непосредственно перед текущей нодой.

БИЗНЕС-ПРАВИЛА

Эскалация приоритета

Правило перебивает модель только вверх. Так принцип «модель предлагает — решает правило» становится конкретным.

МОДЕЛЬ СКАЗАЛА

low



в тексте — «утечка
персональных данных»

ПРАВИЛО ПОСТАВИЛО

critical

Понизить приоритет правило не может — только повысить.

БЕЗОПАСНОСТЬ

Защита от инъекций — в два слоя

Заявка может попытаться переписать инструкции модели. Один слой защиты ненадёжен, поэтому их два.

СЛОЙ 1 · ПРОМПТ

Разметка данными

Текст заявки оборачивается маркерами — модель видит границу между данными и инструкцией.

```
<<<ЗАЯВКА_НАЧАЛО>>>
```

```
Имя: ...
```

```
Контакт: ...
```

```
Текст обращения:
```

```
...
```

```
<<<ЗАЯВКА_КОНЕЦ>>>
```

СЛОЙ 2 · КОД

Детерминированный флаг

Валидация независимо от модели помечает подозрительные заявки — проверка не полагается на LLM.

```
injection_suspected: true
```

Работающий workflow обработки заявок — протестированный и готовый для портфолио.